```
1   //////// ACTIVITY SELECTION PROBLEM ////////
2
3   // Given a set of activities with start and finish times,
4   // select the maximum number of activities that don't overlap.
5
6   struct Activity {
7       int start
8       int finish
9   }
10
11  ActivitySelection(act []Activity, n int) []int:
12      // Sort activities by finish time
13      sort(act, key=lambda x: x.finish)   // O(n log n)
14
15      selected = []
16      i = 1           // pick the first activity
17      selected.append(i)
18
19      for j = 2 to n:     // O(n)
20          if act.start[j] >= act.finish[i]:
21              selected.append(j)
22              i = j
23
24      return selected
```

📘 Pseudocode (Using Min-Heap / Priority Queue)

Given: characters `c[i]` with frequencies `f[i]`.

sql

Copy code

```
Huffman(c[1..n], f[1..n]):
    create a min-heap Q
    for i = 1 to n:
        create a leaf node for c[i]
        insert node into Q with priority f[i]

    while Q.size > 1:
        left  = Q.extractMin()
        right = Q.extractMin()

        newNode.frequency = left.frequency + right.frequency
        newNode.left  = left
        newNode.right = right

        Q.insert(newNode)

    return Q.extractMin()   // root of Huffman tree
```

To generate the codes:

scss

Copy code

```
GenerateCodes(node, code):
    if node is a leaf:
        assign code to node.character
        return

    GenerateCodes(node.left,  code + "0")
    GenerateCodes(node.right, code + "1")
```

📘 Pseudocode (Fractional Knapsack)

Given:

- `value[i]` — profit
- `weight[i]` — weight
- `W` — knapsack capacity

java

Copy code

```
FractionalKnapsack(value[1..n], weight[1..n], W):
    for i = 1 to n:
        ratio[i] = value[i] / weight[i]

    sort items by ratio in descending order

    totalValue = 0

    for i = 1 to n:
        if W == 0:
            break

        if weight[i] <= W:
            // take whole item
            W = W - weight[i]
            totalValue = totalValue + value[i]
        else:
            // take fraction
            fraction = W / weight[i]
            totalValue = totalValue + value[i] * fraction
            W = 0

    return totalValue
```

📘 Pseudocode (0/1 Knapsack – DP Table)

Given:

- `value[i]` — profit
- `weight[i]` — weight
- `W` — capacity
- `n` — number of items

mathematica

Copy code

```
Knapsack01(value[1..n], weight[1..n], W):
    create DP table K[0..n][0..W]

    for w = 0 to W:
        K[0][w] = 0

    for i = 1 to n:
        for w = 0 to W:
            if weight[i] <= w:
                K[i][w] = max(
                              value[i] + K[i-1][w - weight[i]],
                              K[i-1][w]
                             )
            else:
                K[i][w] = K[i-1][w]

    return K[n][W]
```

📘 Pseudocode (Bottom-Up DP)

sql

Copy code

```
RodCutting(price[1..n], n):
    create array dp[0..n]
    dp[0] = 0

    for i = 1 to n:
        maxValue = -∞
        for j = 1 to i:
            maxValue = max(maxValue, price[j] + dp[i - j])
        dp[i] = maxValue

    return dp[n]
```

📘 Pseudocode (Bottom-Up DP)

java

Copy code

```
LCS(X[1..n], Y[1..m]):
    create table dp[0..n][0..m]

    for i = 0 to n:
        dp[i][0] = 0
    for j = 0 to m:
        dp[0][j] = 0

    for i = 1 to n:
        for j = 1 to m:
            if X[i] == Y[j]:
                dp[i][j] = 1 + dp[i-1][j-1]
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[n][m]
```

📘 Pseudocode (BFS using Adjacency List)

sql

Copy code

```
BFS(G, s):
    for each vertex v in G:
        visited[v] = false

    create an empty queue Q
    visited[s] = true
    enqueue(Q, s)

    while Q is not empty:
        u = dequeue(Q)
        visit(u)

        for each v in Adj[u]:
            if visited[v] == false:
                visited[v] = true
                enqueue(Q, v)
```

📘 Pseudocode (Recursive DFS)

scss Copy code

```
DFS(G, u):
    visited[u] = true
    visit(u)

    for each v in Adj[u]:
        if visited[v] == false:
            DFS(G, v)
```

To run DFS for whole graph:

css Copy code

```
for each vertex u in G:
    if visited[u] == false:
        DFS(G, u)
```

Pseudocode (Using Min-Heap)

sql

Copy code

```
Dijkstra(G, source):
    for each vertex v in G:
        dist[v] = ∞
        visited[v] = false

    dist[source] = 0
    create min-priority queue PQ
    insert (0, source) into PQ

    while PQ is not empty:
        (d, u) = extractMin(PQ)

        if visited[u] == true:
            continue

        visited[u] = true

        for each (v, w) in Adj[u]:
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                insert (dist[v], v) into PQ
```

Edge Relaxation Formula

markdown

Copy code

```
if dist[v] > dist[u] + weight(u, v)
    dist[v] = dist[u] + weight(u, v)
```

📘 Pseudocode

rust

Copy code

```
BellmanFord(G, source):
    for each vertex v in G:
        dist[v] = ∞

    dist[source] = 0

    for i = 1 to V-1:
        for each edge (u, v, w) in G:
            if dist[u] ≠ ∞ AND dist[v] > dist[u] + w:
                dist[v] = dist[u] + w

    // Check for negative cycle
    for each edge (u, v, w) in G:
        if dist[u] ≠ ∞ AND dist[v] > dist[u] + w:
            report "Negative weight cycle exists"
```

🧠 Edge Relaxation Formula

lua

Copy code

```
dist[v] = min(dist[v], dist[u] + weight(u, v))
```

Pseudocode

yaml

Copy code

```
FloydWarshall(G):
    for i = 1 to V:
        for j = 1 to V:
            if i == j:
                dist[i][j] = 0
            else if edge(i, j) exists:
                dist[i][j] = weight(i, j)
            else:
                dist[i][j] = ∞

    for k = 1 to V:
        for i = 1 to V:
            for j = 1 to V:
                if dist[i][j] > dist[i][k] + dist[k][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
```

Pseudocode (Prim using Min-Heap)

vbnet

Copy code

```
Prim(G):
    for each vertex v:
        key[v] = ∞
        parent[v] = NIL
        inMST[v] = false

    key[0] = 0
    PQ = min-priority queue of (key, vertex)

    while PQ not empty:
        u = extractMin(PQ)
        inMST[u] = true

        for each (v, w) in Adj[u]:
            if inMST[v] == false AND w < key[v]:
                key[v] = w
                parent[v] = u
                insert (key[v], v) into PQ
```

Pseudocode

scss

Copy code

```
Kruskal(G):
    MST = empty set
    sort edges by weight

    for each vertex v:
        MakeSet(v)

    for each edge (u, v, w):
        if Find(u) ≠ Find(v):
            MST.add(edge)
            Union(u, v)
```